INSTITUTE
OF LAW,
POLITICS AND
DEVELOPMENT



Sant'Anna
School of Advanced Studies – Pisa

Data Analysis in R using Agentic AI Assistants

Session 2: Data management

Prof. Kevin Koehler
kevin.koehler@santannapisa.it

1. Housekeeping



Sant'Anna
School of Advanced Studies – Pisa

1.1 Two sessions have moved

	Was	Now
Session 3 Visualization	Mon 2 Nov, 09:00–11:00	**Fri 6 Nov, 16:00–18:00** Palazzo Boyl, Aula "Talento all'Opera"
Session 4 R programming	Mon 9 Nov, 16:00–18:00	**Fri 13 Nov, 14:00–16:00** Sede Centrale, Aula 5

No class on Monday 2 or Monday 9 November. From Session 5 (Mon 16 Nov, 16:00–18:00) we are back to the usual schedule. The syllabus and the GitHub page are updated.

2. Last week

2.1 Your code files

Open the script the assistant wrote for you last week.

- Any **specific questions** about your file? A line you could not explain, an error, an output you did not trust?

While we read, ask of every line: *what does this do to the data?* And of the whole file: *could I explain this to someone else?*

2.1 Your code files

Open the script the assistant wrote for you last week.

- Any **specific questions** about your file? A line you could not explain, an error, an output you did not trust?
- Then one of you, picked at random, puts their file **on the screen**, and we read it together.

While we read, ask of every line: *what does this do to the data?* And of the whole file: *could I explain this to someone else?*

2.2 A new `AGENTS.md`

The scripts were too complicated. I have changed the standing instructions. The assistant now has to:

- write **simple** code: no functions of its own, no loops over variables, no clever one-liners

Today: download the new `AGENTS.md` from the course GitHub and replace the one in your project folder.

2.2 A new `AGENTS.md`

The scripts were too complicated. I have changed the standing instructions. The assistant now has to:

- write **simple** code: no functions of its own, no loops over variables, no clever one-liners
- treat each variable in its own block, even if that repeats code

Today: download the new `AGENTS.md` from the course GitHub and replace the one in your project folder.

2.2 A new `AGENTS.md`

The scripts were too complicated. I have changed the standing instructions. The assistant now has to:

- write **simple** code: no functions of its own, no loops over variables, no clever one-liners
- treat each variable in its own block, even if that repeats code
- print the number of rows after every step that can change it

Today: download the new `AGENTS.md` from the course GitHub and replace the one in your project folder.

2.2 A new `AGENTS.md`

The scripts were too complicated. I have changed the standing instructions. The assistant now has to:

- write **simple** code: no functions of its own, no loops over variables, no clever one-liners
- treat each variable in its own block, even if that repeats code
- print the number of rows after every step that can change it
- flag codes like 98 and 99 and **ask you** what to do with them

Today: download the new `AGENTS.md` from the course GitHub and replace the one in your project folder.

2.3 Same prompt, new rules

I run last week's prompt again, in a fresh session (`/new`):

To OpenCode

Create an R script that produces summary statistics for the Tunisia survey data.

Compare it with last week's script:

- Did it define functions of its own? Loop over anything?

2.3 Same prompt, new rules

I run last week's prompt again, in a fresh session (`/new`):

To OpenCode

Create an R script that produces summary statistics for the Tunisia survey data.

Compare it with last week's script:

- Did it define functions of its own? Loop over anything?
- Does it print row counts? Did it ask about 98 and 99?

2.3 Same prompt, new rules

I run last week's prompt again, in a fresh session (`/new`):

To OpenCode

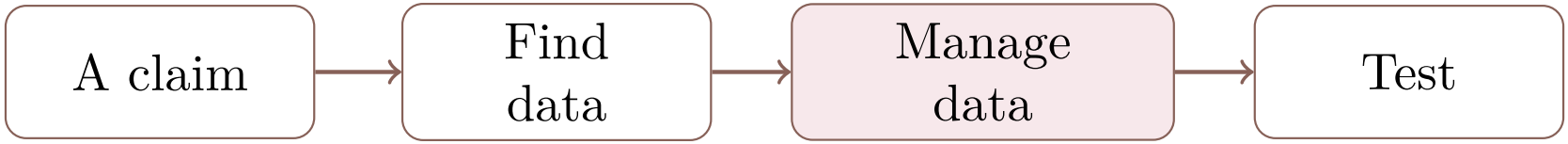
Create an R script that produces summary statistics for the Tunisia survey data.

Compare it with last week's script:

- Did it define functions of its own? Loop over anything?
- Does it print row counts? Did it ask about 98 and 99?
- Can you now explain every line?

3. What is data management?

3.1 Everything between "I found data" and "I can test my claim"

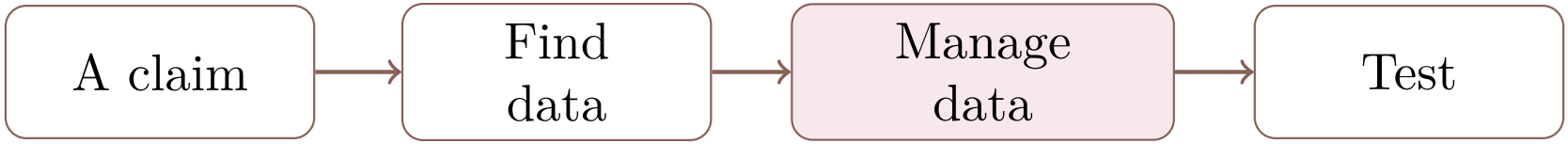


clean, combine, reshape, check

Very few questions can be answered with one dataset as you download it. Usually you need to:

- **combine** information from different sources (conflict from one, income from another)

3.1 Everything between "I found data" and "I can test my claim"

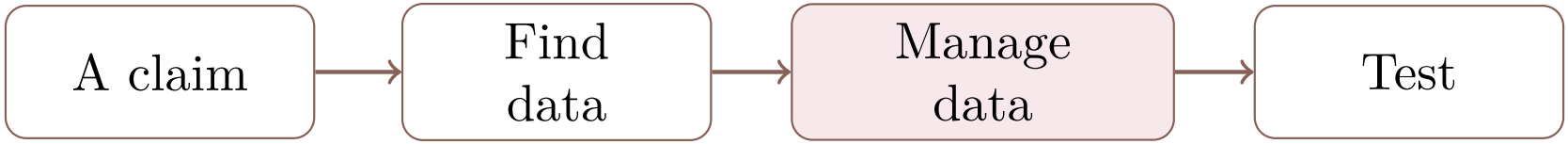


clean, combine, reshape, check

Very few questions can be answered with one dataset as you download it. Usually you need to:

- **combine** information from different sources (conflict from one, income from another)
- bring each source into the **same shape** (one row per country and year)

3.1 Everything between "I found data" and "I can test my claim"



clean, combine, reshape, check

Very few questions can be answered with one dataset as you download it. Usually you need to:

- **combine** information from different sources (conflict from one, income from another)
- bring each source into the **same shape** (one row per country and year)
- **check** that nothing was lost, duplicated or changed on the way

3.2 Why is it complicated?

Every dataset is built by someone else, for their own purposes:

- **Different units.** One row per event, per conflict-year, per country-year, per respondent.

Each of these needs a decision. If you do not make it, the assistant makes it for you, and usually does not tell you.

3.2 Why is it complicated?

Every dataset is built by someone else, for their own purposes:

- **Different units.** One row per event, per conflict-year, per country-year, per respondent.
- **Different names for the same thing.** Côte d'Ivoire, Ivory Coast, Cote d'Ivoire, CIV, 437.

Each of these needs a decision. If you do not make it, the assistant makes it for you, and usually does not tell you.

3.2 Why is it complicated?

Every dataset is built by someone else, for their own purposes:

- **Different units.** One row per event, per conflict-year, per country-year, per respondent.
- **Different names for the same thing.** Côte d'Ivoire, Ivory Coast, Cote d'Ivoire, CIV, 437.
- **Different coverage.** Years, and countries that no longer exist (USSR, Yugoslavia) or are not recognised by everyone (Kosovo, Taiwan).

Each of these needs a decision. If you do not make it, the assistant makes it for you, and usually does not tell you.

3.2 Why is it complicated?

Every dataset is built by someone else, for their own purposes:

- **Different units.** One row per event, per conflict-year, per country-year, per respondent.
- **Different names for the same thing.** Côte d'Ivoire, Ivory Coast, Cote d'Ivoire, CIV, 437.
- **Different coverage.** Years, and countries that no longer exist (USSR, Yugoslavia) or are not recognised by everyone (Kosovo, Taiwan).
- **Different shapes.** Years in rows, or years in columns.

Each of these needs a decision. If you do not make it, the assistant makes it for you, and usually does not tell you.

3.2 Why is it complicated?

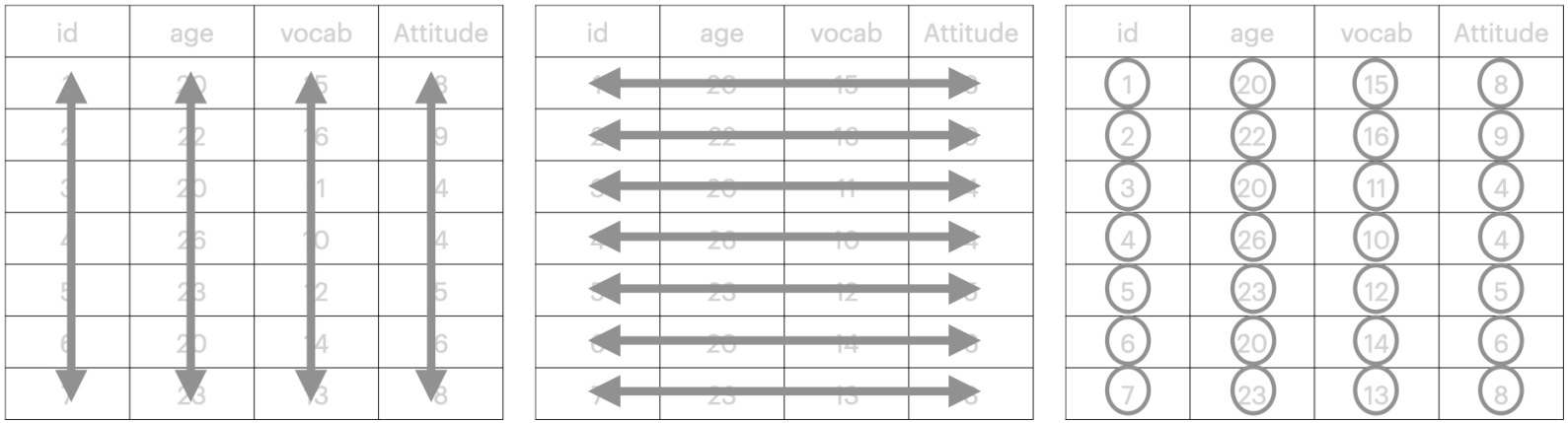
Every dataset is built by someone else, for their own purposes:

- **Different units.** One row per event, per conflict-year, per country-year, per respondent.
- **Different names for the same thing.** Côte d'Ivoire, Ivory Coast, Cote d'Ivoire, CIV, 437.
- **Different coverage.** Years, and countries that no longer exist (USSR, Yugoslavia) or are not recognised by everyone (Kosovo, Taiwan).
- **Different shapes.** Years in rows, or years in columns.
- **Different codes for "missing".** Blank, `NA`, -99, 98, 99.

Each of these needs a decision. If you do not make it, the assistant makes it for you, and usually does not tell you.

4. Tidy data

4.1 What we want: tidy data



Each **variable**
a column

Each **observation**
a row

Each **value**
a cell

What counts as "an observation" depends on your **unit of analysis**: a respondent, a party in an election, a country in a year.

4.2 Messy or tidy?

Messy: the year is hidden in the column names

Party	**2014 %**	**2019 %**
Nidaa Tounes	37.56	1.51
Ennahda	27.79	19.63

Tidy: party, year and vote are each a column

party	**year**	**vote**
Nidaa Tounes	2014	37.56
Nidaa Tounes	2019	1.51
Ennahda	2014	27.79
Ennahda	2019	19.63

The messy version is easier to **read**. The tidy version is easier to **analyse**: "average vote by year" is one step.

4.3 Is this tidy?

A

country	gdp_19_20
Tunisia	3.8 / 3.5
Egypt	3.0 / 3.1

B

conflict	location
101	Russia, Ukraine
102	Mali

C

country	year	war
Mali	2019	1
Mali	2019	1
Peru	2019	0

- **A:** two values in one cell, and the year hidden in the name.

Illustrative numbers.

4.3 Is this tidy?

A

country	gdp_19_20
Tunisia	3.8 / 3.5
Egypt	3.0 / 3.1

B

conflict	location
101	Russia, Ukraine
102	Mali

C

country	year	war
Mali	2019	1
Mali	2019	1
Peru	2019	0

- **A:** two values in one cell, and the year hidden in the name.
- **B:** two countries in one cell. Which country does the conflict belong to?

Illustrative numbers.

4.3 Is this tidy?

A

country	gdp_19_20
Tunisia	3.8 / 3.5
Egypt	3.0 / 3.1

B

conflict	location
101	Russia, Ukraine
102	Mali

C

country	year	war
Mali	2019	1
Mali	2019	1
Peru	2019	0

- **A:** two values in one cell, and the year hidden in the name.
- **B:** two countries in one cell. Which country does the conflict belong to?
- **C:** looks tidy, but Mali-2019 appears twice. If the unit is the country-year, that is one observation counted twice.

Illustrative numbers.

5. Merging

5.1 Merging: adding columns from another table

Conflict data

country	conflict
MLI	1
PER	0

+

Income data

country	gdp
PER	6.6
MLI	0.9

=

Merged

country	conflict	gdp
MLI	1	0.9
PER	0	6.6

Rows are matched on an **identifier** (the "key"), not on their position in the file.

- The key follows the **unit of analysis**: country, country-year, respondent, party, MP, conflict.

5.1 Merging: adding columns from another table

Conflict data

country	conflict
MLI	1
PER	0

+

Income data

country	gdp
PER	6.6
MLI	0.9

=

Merged

country	conflict	gdp
MLI	1	0.9
PER	0	6.6

Rows are matched on an **identifier** (the "key"), not on their position in the file.

- The key follows the **unit of analysis**: country, country-year, respondent, party, MP, conflict.
- For country-years the key is **two** columns: country **and** year.

5.1 Merging: adding columns from another table

Conflict data

country	conflict
MLI	1
PER	0

+

Income data

country	gdp
PER	6.6
MLI	0.9

=

Merged

country	conflict	gdp
MLI	1	0.9
PER	0	6.6

Rows are matched on an **identifier** (the "key"), not on their position in the file.

- The key follows the **unit of analysis**: country, country-year, respondent, party, MP, conflict.
- For country-years the key is **two** columns: country **and** year.
- It only works if the key is written **the same way** in both tables.

5.2 The same country, six ways

UCDP name	World Bank name	COW	G&W	ISO-3c	ISO-3n
Ivory Coast	Cote d'Ivoire	437	437	CIV	384
DR Congo (Zaire)	Congo, Dem. Rep.	490	490	COD	180
Myanmar (Burma)	Myanmar	775	775	MMR	104
Germany	Germany	255	260	DEU	276

Names as the producers write them; codes from Correlates of War (COW), Gleditsch & Ward (G&W, used by UCDP) and ISO 3166 (letters and numbers). Red: the same country, different codes.

5.3 Names and codes

- **Names** are the worst key: spelling, accents, “the”, former names.

5.3 Names and codes

- **Names** are the worst key: spelling, accents, “the”, former names.
- **Codes** are better, but there are several systems, and each dataset picks one.

5.3 Names and codes

- **Names** are the worst key: spelling, accents, “the”, former names.
- **Codes** are better, but there are several systems, and each dataset picks one.
- Even codes differ: unified Germany is 255 in COW but 260 in G&W.

5.3 Names and codes

- **Names** are the worst key: spelling, accents, “the”, former names.
- **Codes** are better, but there are several systems, and each dataset picks one.
- Even codes differ: unified Germany is 255 in COW but 260 in G&W.
- So the first step is often to **create a common code**. In R, the `countrycode` package converts between systems. Then check what it could not convert.

5.4 When there is no standard code

For parties, armed groups, MPs, villages or companies there is usually no common code. What can we do?

- **Look for a linking table** someone already built. Party Facts, for example, links party IDs across many party datasets.

Either way: the result is a guess. Check a sample of the matches by hand, and look at what did **not** match.

5.4 When there is no standard code

For parties, armed groups, MPs, villages or companies there is usually no common code. What can we do?

- **Look for a linking table** someone already built. Party Facts, for example, links party IDs across many party datasets.
- **Fuzzy matching:** match names that are *similar*, measured by how many letters you must change to get from one to the other. "Nidaa Tounes" and "Nidaa Tounès" match. So, wrongly, might "Niger" and "Nigeria".

Either way: the result is a guess. Check a sample of the matches by hand, and look at what did **not** match.

5.4 When there is no standard code

For parties, armed groups, MPs, villages or companies there is usually no common code. What can we do?

- **Look for a linking table** someone already built. Party Facts, for example, links party IDs across many party datasets.
- **Fuzzy matching:** match names that are *similar*, measured by how many letters you must change to get from one to the other. "Nidaa Tounes" and "Nidaa Tounès" match. So, wrongly, might "Niger" and "Nigeria".
- **Probabilistic record linkage:** combine several fields (name, birth year, district) and estimate how likely two records are the same person. In R: `fastLink` (Enamorado, Fifield & Imai 2019).

Either way: the result is a guess. Check a sample of the matches by hand, and look at what did **not** match.

5.5 Two things a merge can do to your rows

- **Lose them.** Rows with no match get `NA`, or disappear, depending on the type of join. Look at the rows that did not match: they are rarely random (small states, former states, disputed territories).

The checks, whoever wrote the code:

- Rows **before** and **after**. Should the number change?
- Is the key **unique** in each table? One row per country-year?
- **Which** rows found no match, and why?

5.5 Two things a merge can do to your rows

- **Lose them.** Rows with no match get `NA`, or disappear, depending on the type of join. Look at the rows that did not match: they are rarely random (small states, former states, disputed territories).
- **Multiply them.** If the key appears twice in the second table, every match is copied twice. A survey of 1,000 people becomes 2,000. Averages may not change, so nothing looks wrong.

The checks, whoever wrote the code:

- Rows **before** and **after**. Should the number change?
- Is the key **unique** in each table? One row per country-year?
- **Which** rows found no match, and why?

6. Reshaping

6.1 Reshaping (transposing): wide ↔ long

Wide: one row per country

country	**2019**	**2020**
Tunisia	3.8	3.5
Egypt	3.0	3.1

`pivot_longer()` →

← `pivot_wider()`

Long: one row per country-year

country	year	gdp
Tunisia	2019	3.8
Tunisia	2020	3.5
Egypt	2019	3.0
Egypt	2020	3.1

- Going long, the **column names** (2019, 2020) become **values** of a new column, `year`. Each coloured column becomes a block of rows.

Illustrative numbers (GDP per capita, thousand US$).

6.1 Reshaping (transposing): wide ↔ long

Wide: one row per country

country	**2019**	**2020**
Tunisia	3.8	3.5
Egypt	3.0	3.1

`pivot_longer()` →

← `pivot_wider()`

Long: one row per country-year

country	year	gdp
Tunisia	2019	3.8
Tunisia	2020	3.5
Egypt	2019	3.0
Egypt	2020	3.1

- Going long, the **column names** (2019, 2020) become **values** of a new column, `year`. Each coloured column becomes a block of rows.
- Same information, different shape: 2 countries × 2 years = 4 rows.

Illustrative numbers (GDP per capita, thousand US$).

6.2 Reshaping: what to check

- Data downloaded from the World Bank comes **wide**, one column per year. To merge it with country-year data, it must go long first.

6.2 Reshaping: what to check

- Data downloaded from the World Bank comes **wide**, one column per year. To merge it with country-year data, it must go long first.
- **Count the rows.** Wide to long: countries $\times$ years. If you have 200 countries and 30 years, you should get 6,000 rows.

6.2 Reshaping: what to check

- Data downloaded from the World Bank comes **wide**, one column per year. To merge it with country-year data, it must go long first.
- **Count the rows.** Wide to long: countries × years. If you have 200 countries and 30 years, you should get 6,000 rows.
- **Is `year` a number?** Column names are text, so after `pivot_longer()` the year often is too ("2019", or even "YR2019").

6.2 Reshaping: what to check

- Data downloaded from the World Bank comes **wide**, one column per year. To merge it with country-year data, it must go long first.
- **Count the rows.** Wide to long: countries × years. If you have 200 countries and 30 years, you should get 6,000 rows.
- **Is `year` a number?** Column names are text, so after `pivot_longer()` the year often is too ("2019", or even "YR2019").
- **Long to wide** only works if each country-year appears once. Otherwise R has two values for one cell, and puts them in a list or warns.

6.2 Reshaping: what to check

- Data downloaded from the World Bank comes **wide**, one column per year. To merge it with country-year data, it must go long first.
- **Count the rows.** Wide to long: countries × years. If you have 200 countries and 30 years, you should get 6,000 rows.
- **Is `year` a number?** Column names are text, so after `pivot_longer()` the year often is too ("2019", or even "YR2019").
- **Long to wide** only works if each country-year appears once. Otherwise R has two values for one cell, and puts them in a list or warns.
- Sometimes you need **both**: reshape one dataset, aggregate another, *then* merge.

7. Lab: test a claim

Since the end of the Cold War, internal armed conflict has been concentrated in poorer countries.

Is that true? Your job, in pairs, in five steps:

1. **Find** the data and read the codebooks without AI
2. **Prompt** the assistant to merge them
3. **Inspect** the merged data
4. **Read** the code that produced it
5. **Design** a test of the claim, and prompt it

7.2 Step 1: find the data (no AI)

Conflict: **UCDP/PRIO Armed Conflict Dataset** (ucdp.uu.se).
Income: **World Bank, GDP per capita** (data.worldbank.org).
Download the latest versions, open the codebooks, and write down:

	UCDP	**World Bank**
Which version?		
One row = ? (unit of analysis)		
How is a country identified? Name? Which code?		
Which years?		
Which variable will you use? How is it measured?		
How is "missing" coded?		

Reading codebooks is your job: the assistant will happily describe a variable that does not exist.

7.3 Step 2: prompt the merge

Write a prompt that tells the assistant to merge your two datasets into **one table, one row per country-year**.

Your prompt has to say:

- which **files**, and where they are

Look at your Step 1 table first. Do both datasets use the same identifier? Is each already one row per country-year? If not, your prompt must say what should happen **before** the merge.

Save the result as `output/conflict_income.csv`.

7.3 Step 2: prompt the merge

Write a prompt that tells the assistant to merge your two datasets into **one table, one row per country-year**.

Your prompt has to say:

- which **files**, and where they are
- which column **identifies the country** in each file, and which **code system** it uses

Look at your Step 1 table first. Do both datasets use the same identifier? Is each already one row per country-year? If not, your prompt must say what should happen **before** the merge.

Save the result as `output/conflict_income.csv`.

7.3 Step 2: prompt the merge

Write a prompt that tells the assistant to merge your two datasets into **one table, one row per country-year**.

Your prompt has to say:

- which **files**, and where they are
- which column **identifies the country** in each file, and which **code system** it uses
- which **years** you want

Look at your Step 1 table first. Do both datasets use the same identifier? Is each already one row per country-year? If not, your prompt must say what should happen **before** the merge.

Save the result as `output/conflict_income.csv`.

7.3 Step 2: prompt the merge

Write a prompt that tells the assistant to merge your two datasets into **one table, one row per country-year**.

Your prompt has to say:

- which **files**, and where they are
- which column **identifies the country** in each file, and which **code system** it uses
- which **years** you want
- what **one row** of the final table should be

Look at your Step 1 table first. Do both datasets use the same identifier? Is each already one row per country-year? If not, your prompt must say what should happen **before** the merge.

Save the result as `output/conflict_income.csv`.

7.4 Step 3: inspect the data

Open the merged file. **Without asking the assistant:**

1. **Did the merge work?** How many rows? How many countries? How many years? Does that match countries × years?

Found a problem? Write a second prompt that fixes it, and check again.

7.4 Step 3: inspect the data

Open the merged file. **Without asking the assistant:**

1. **Did the merge work?** How many rows? How many countries? How many years? Does that match countries × years?
2. **What did not match?** Which countries have conflict data but no income, or the reverse? Why those?

Found a problem? Write a second prompt that fixes it, and check again.

7.4 Step 3: inspect the data

Open the merged file. **Without asking the assistant:**

1. **Did the merge work?** How many rows? How many countries? How many years? Does that match countries × years?
2. **What did not match?** Which countries have conflict data but no income, or the reverse? Why those?
3. **Is it tidy?** One row per country-year? Any country-year twice? Any cell with two values?

Found a problem? Write a second prompt that fixes it, and check again.

7.4 Step 3: inspect the data

Open the merged file. **Without asking the assistant:**

1. **Did the merge work?** How many rows? How many countries? How many years? Does that match countries × years?
2. **What did not match?** Which countries have conflict data but no income, or the reverse? Why those?
3. **Is it tidy?** One row per country-year? Any country-year twice? Any cell with two values?
4. **Is everything there?** A country with no conflict in a year: is it coded 0, or `NA`? Those are not the same thing.

Found a problem? Write a second prompt that fixes it, and check again.

7.5 What usually goes wrong

- **Different codes.** UCDP uses Gleditsch & Ward codes; the World Bank uses ISO-3 letters. Without a conversion, almost nothing matches, or names match for some countries and silently fail for others.

7.5 What usually goes wrong

- **Different codes.** UCDP uses Gleditsch & Ward codes; the World Bank uses ISO-3 letters. Without a conversion, almost nothing matches, or names match for some countries and silently fail for others.
- **Wrong unit.** UCDP has one row per **conflict**-year. A country with two conflicts in a year appears twice, and the merge copies its income twice.

7.5 What usually goes wrong

- **Different codes.** UCDP uses Gleditsch & Ward codes; the World Bank uses ISO-3 letters. Without a conversion, almost nothing matches, or names match for some countries and silently fail for others.
- **Wrong unit.** UCDP has one row per **conflict**-year. A country with two conflicts in a year appears twice, and the merge copies its income twice.
- **Wrong shape.** The World Bank file has one column per year. It must go long before the merge.

7.5 What usually goes wrong

- **Different codes.** UCDP uses Gleditsch & Ward codes; the World Bank uses ISO-3 letters. Without a conversion, almost nothing matches, or names match for some countries and silently fail for others.
- **Wrong unit.** UCDP has one row per **conflict**-year. A country with two conflicts in a year appears twice, and the merge copies its income twice.
- **Wrong shape.** The World Bank file has one column per year. It must go long before the merge.
- **Missing is not zero.** Countries without conflict are not in the conflict data at all. After the merge they are `NA`; for our claim they should be 0.

7.5 What usually goes wrong

- **Different codes.** UCDP uses Gleditsch & Ward codes; the World Bank uses ISO-3 letters. Without a conversion, almost nothing matches, or names match for some countries and silently fail for others.
- **Wrong unit.** UCDP has one row per **conflict**-year. A country with two conflicts in a year appears twice, and the merge copies its income twice.
- **Wrong shape.** The World Bank file has one column per year. It must go long before the merge.
- **Missing is not zero.** Countries without conflict are not in the conflict data at all. After the merge they are `NA`; for our claim they should be 0.
- **Lost countries.** Kosovo, Taiwan, former states: not in both datasets.

7.6 Step 4: read the code

Open the script the assistant wrote. With your neighbour:

1. Find the line that **converts country codes**. Which systems, from and to? What happens to countries it cannot convert?

If you cannot explain a step, you cannot defend the result.

7.6 Step 4: read the code

Open the script the assistant wrote. With your neighbour:

1. Find the line that **converts country codes**. Which systems, from and to? What happens to countries it cannot convert?
2. Find the line that **reshapes** or **aggregates**. What is one row before it, and after?

If you cannot explain a step, you cannot defend the result.

7.6 Step 4: read the code

Open the script the assistant wrote. With your neighbour:

1. Find the line that **converts country codes**. Which systems, from and to? What happens to countries it cannot convert?
2. Find the line that **reshapes** or **aggregates**. What is one row before it, and after?
3. Find the **merge**. Which type of join? What happens to rows without a match?

If you cannot explain a step, you cannot defend the result.

7.6 Step 4: read the code

Open the script the assistant wrote. With your neighbour:

1. Find the line that **converts country codes**. Which systems, from and to? What happens to countries it cannot convert?
2. Find the line that **reshapes** or **aggregates**. What is one row before it, and after?
3. Find the **merge**. Which type of join? What happens to rows without a match?
4. Pick the line you understand least. Look up the function with `?` and explain it in your own words.

If you cannot explain a step, you cannot defend the result.

7.7 Step 5: design the test

Before you prompt anything, decide:

- What does **"concentrated"** mean? A higher share of conflict-years among poorer countries? A lower average income in conflict country-years?

Then write the prompt. Ask for a table or a chart you can read, not just a number.

7.7 Step 5: design the test

Before you prompt anything, decide:

- What does **"concentrated"** mean? A higher share of conflict-years among poorer countries? A lower average income in conflict country-years?
- What is **poorer**? Income groups, quartiles, or GDP per capita as a number?

Then write the prompt. Ask for a table or a chart you can read, not just a number.

7.7 Step 5: design the test

Before you prompt anything, decide:

- What does **"concentrated"** mean? A higher share of conflict-years among poorer countries? A lower average income in conflict country-years?
- What is **poorer**? Income groups, quartiles, or GDP per capita as a number?
- Which **years** count as "since the end of the Cold War"?

Then write the prompt. Ask for a table or a chart you can read, not just a number.

7.7 Step 5: design the test

Before you prompt anything, decide:

- What does **"concentrated"** mean? A higher share of conflict-years among poorer countries? A lower average income in conflict country-years?
- What is **poorer**? Income groups, quartiles, or GDP per capita as a number?
- Which **years** count as "since the end of the Cold War"?
- Could the pattern run the other way? Conflict **makes** countries poorer. Which year's income should you use?

Then write the prompt. Ask for a table or a chart you can read, not just a number.

7.8 What did we find?

- Is the claim true? How sure are you?

Data management is not a technical step before the analysis. It **is** part of the analysis.

7.8 What did we find?

- Is the claim true? How sure are you?
- Which decision changed the answer the most: the code system, the unit, missing as zero, the years, the income measure?

Data management is not a technical step before the analysis. It **is** part of the analysis.

7.8 What did we find?

- Is the claim true? How sure are you?
- Which decision changed the answer the most: the code system, the unit, missing as zero, the years, the income measure?
- Where did the assistant make a decision you had not asked for?

Data management is not a technical step before the analysis. It **is** part of the analysis.

8. Wrapping up

8.1 For next time

Read: Kennedy & Waggoner (2021), Chapter 4

Do:

- Replace `AGENTS.md` with the new version
- Keep your merged file, your script and your prompts
- Add one line to `log.md`: something the assistant got wrong today, and how you caught it

Session 3 (Friday 6 November, 16:00–18:00, Palazzo Boyl, Aula TaO): visualization — and the ways a chart can look right while showing the wrong thing.